Módulo 4

Tipos de Datos Básicos

Curso: TypeScript

75 min

Objetivos de Aprendizaje

- Comprender los tipos primitivos básicos de TypeScript (**string**, **number**, **boolean**)
- Utilizar **arrays** y **tuplas** para almacenar colecciones de datos
- Implementar **enums** para definir conjuntos de valores constantes
- Distinguir entre **any**, **void**, **null**, **undefined** y **never**
- Aplicar **type annotations** de manera efectiva en el código

1. Tipos Primitivos

string, number, boolean

Tipos Primitivos Básicos

TypeScript soporta tres tipos primitivos fundamentales:

- **string**: Para texto
- **number**: Para números
- **boolean**: Para valores verdadero/falso

Ejemplo: string, number, boolean

```
// Tipo string - texto
let taskTitle: string = "Completar informe";
let userName: string = "Juan Pérez";

// Tipo number - números
let taskPriority: number = 1;
let completedTasks: number = 42;
let averageTime: number = 3.5;

// Tipo boolean - verdadero o falso
let isCompleted: boolean = false;
let isUrgent: boolean = true;
```

Caso 1: Sistema de Tareas

```
// Aplicando tipos primitivos al Caso 1
let title: string = "Revisar código";
let effort: number = 8;
let isDone: boolean = false;
```

Caso 1: Sistema de Gestión de Tareas

2. Arrays

Colecciones del mismo tipo

Arrays Tipados

Los arrays en TypeScript pueden tiparse de dos formas:

- **Sintaxis 1:** `tipo[]`
- **Sintaxis 2:** `Array<tipo>`

Ejemplo: Arrays

```
// Sintaxis 1: tipo[]
let tasks: string[] = ["Task 1", "Task 2", "Task 3"];

// Sintaxis 2: Array<tipo>
let priorities: Array<number> = [1, 2, 3, 4, 5];

// Arrays de objetos
interface Task {
  id: string;
  title: string;
}

let taskList: Task[] = [
  { id: "1", title: "Task 1" },
  { id: "2", title: "Task 2" }
];
```

3. Tuplas

Arrays con tipos específicos por posición

Tuplas

Las tuplas permiten definir arrays con tipos específicos en cada posición:

```
// Tupla: [string, number]
let userInfo: [string, number] = ["Juan", 30];

// Acceso por índice
console.log(userInfo[0]); // "Juan"
console.log(userInfo[1]); // 30

// Tupla con tipos mixtos
let taskRecord: [string, number, boolean] = ["Task 1", 5, true];
```

Caso 1: Tuplas en Sistema de Tareas

```
// Guardar [id, título, completada]
let taskData: [string, string, boolean] = ["1", "Nueva tarea", false];
```

Tip: Las tuplas son ideales para pares clave-valor o registros pequeños con tipos fijos.

4. Enums

Conjuntos de valores con nombre

Enums

Los enums definen un conjunto de valores con nombre:

```
// Enum numérico
enum Status {
  Pending,    // 0
  InProgress, // 1
  Completed   // 2
}

let taskStatus: Status = Status.Pending;

// Enum con valores de string
enum Priority {
  Low = "low",
  Medium = "medium",
  High = "high"
}

let taskPriority: Priority = Priority.High;
```

Caso 1: Enums para Sistema de Tareas

```
enum TaskStatus {
  Pending = "pending",
  InProgress = "inProgress",
  Completed = "completed"
}

enum TaskPriority {
  Low = 1,
  Medium = 2,
  High = 3
}
```

Best Practice: Usar enums con strings para mejor legibilidad y depuración.

5. Any, Void, Null, Undefined

Tipos especiales

any - Cualquier tipo

Permite cualquier valor sin verificación de tipos:

```
let anything: any = "texto";
anything = 42;
anything = true;
// No produce errores

// Peligro: any desactiva la verificación
let data: any = "hello";
data.toFixed(2); // Error en runtime, no en compilación
```

⚠️ **Cuidado:** Usar `any` elimina los beneficios de TypeScript.

void - Sin retorno

Para funciones que no retornan nada:

```
function logMessage(message: string): void {
  console.log(message);
  // No retorna nada
}
```

null y undefined

```
// strictNullChecks enabled (recomendado)
let nullable: string | null = null;
let optional: number | undefined = undefined;

// strictNullChecks disabled
let anything: string = null; // permite
```

Tip: Habilitar `strictNullChecks` en tsconfig.json para mayor seguridad.

6. Never

Funciones que nunca retornan

never - Nunca retorna

Para funciones que nunca completan o siempre lanzan errores:

```
// Función que lanza error
function throwError(message: string): never {
  throw new Error(message);
}

// Función con loop infinito
function infiniteLoop(): never {
  while (true) {
    // nunca termina
  }
}
```

Nota: `never` es el tipo de retorno cuando una función nunca completa exitosamente.

7. Type Annotations

Anotaciones explícitas de tipo

Anotaciones de Tipo

TypeScript permite agregar tipos explícitos:

```
// Variables
let name: string = "Juan";
let age: number = 25;
let active: boolean = true;

// Parámetros de función
function greet(person: string): string {
  return `Hola, ${person}`;
}

// Valores de retorno
function calculate(x: number, y: number): number {
  return x + y;
}
```

Type Inference

TypeScript infiere el tipo automáticamente:

```
// Type inference (TypeScript infiere el tipo)
let inferred = "hola"; // infiere: string
let inferred2 = 42;    // infiere: number
```

Tip: En la mayoría de los casos, TypeScript infiere correctamente el tipo. Usa anotaciones cuando sea necesario documentar o cuando la inferencia no sea clara.

Lab

Tipos Básicos - Caso 1: Sistema de Tareas

Objetivo del Lab

Objetivo: Definir tipos para el Sistema de Gestión de Tareas

Duración: 25 minutos

Paso 1: Crear archivo de tipos

```
// src/types/task.ts

// Enum para estados de tarea
export enum TaskStatus {
  Pending = "pending",
  InProgress = "inProgress",
  Completed = "completed"
}

// Enum para prioridades
export enum TaskPriority {
  Low = 1,
  Medium = 2,
  High = 3
}

// Interfaz Task
export interface Task {
  id: string;
```

Paso 2: Crear instancia de tarea

```
// src/index.ts
import { Task, TaskStatus, TaskPriority } from "./types/task";

const newTask: Task = {
  id: "1",
  title: "Aprender TypeScript",
  description: "Completar el curso de TypeScript",
  status: TaskStatus.Pending,
  priority: TaskPriority.High,
  dueDate: "2024-12-31",
  createdAt: new Date().toISOString()
};

console.log(newTask);
```

Paso 3: Arrays y Tuplas

```
// Arrays de tareas
const tasks: Task[] = [
  newTask,
  {
    id: "2",
    title: "Práctica",
    description: "Hacer ejercicios",
    status: TaskStatus.InProgress,
    priority: TaskPriority.Medium,
    dueDate: null,
    createdAt: new Date().toISOString()
  }
];

// Tupla para metadata
const taskMetadata: [string, number, boolean] = ["tarea-1", 5, false];
```

Verificar

```
# Compilar y ejecutar
npx tsc
node dist/index.js
```

Expected: Debería mostrar los objetos Task en consola sin errores de compilación.

Tips y Gotchas

Errores comunes y mejores prácticas

✅ Tips

- Usa tipos específicos: `string`, `number`, `boolean`
- Enums útiles para valores fijos, pero considera unions de literales
- Arrays: `number[]` o `Array<number>` (equivalentes)
- Tuplas para longitud fija con tipos específicos

⚠️ Gotchas

- `any` desactiva TS completamente - evítalo
- `null` y `undefined` son tipos distintos
- `never` ≠ `void`
- Numeric literals: `type Priority = 1 | 2 | 3`

Ejemplo: any vs unknown

```
// ❌ Gotcha: any desactiva verificación
function process(data: any) { return data.foo; }

// ✅ Solución: unknown + type narrowing
function process(data: unknown) {
  if (typeof data === 'object' && data !== null) {
    return (data as { foo: string }).foo;
  }
  throw new Error('Invalid data');
}
```

Ejercicio Integrador

Caso 2: Sistema de Biblioteca

Objetivo

Objetivo: Definir tipos para el Sistema de Gestión de Biblioteca

Duración: 20 minutos | Trabajo en grupo (2-4 personas)

Paso 1: Tipos para Libro

```
// src/domains/books/types.ts

export enum Genre {
  Fiction = "fiction",
  NonFiction = "nonFiction",
  Science = "science",
  History = "history",
  Technology = "technology"
}

export interface Book {
  isbn: string;
  title: string;
  author: string;
  genre: Genre;
  available: boolean;
  copies: number;
}
```

Paso 2: Tipos para Miembro

```
// src/domains/members/types.ts

export enum MembershipType {
  Standard = "standard",
  Premium = "premium",
  VIP = "vip"
}

export interface Member {
  id: string;
  name: string;
  email: string;
  membershipType: MembershipType;
  joinedDate: string;
  borrowedBooks: string[]; // ISBNs
}
```

Paso 3: Tipos para Préstamo

```
// src/domains/loans/types.ts

export enum LoanStatus {
  Active = "active",
  Returned = "returned",
  Overdue = "overdue"
}

export interface Loan {
  id: string;
  bookIsbn: string;
  memberId: string;
  loanDate: string;
  dueDate: string;
  returnDate: string | null;
  status: LoanStatus;
}
```

Paso 4: Usar todos los tipos

```
// src/index.ts
import { Book, Genre } from "./domains/books/types";
import { Member, MembershipType } from "./domains/members/types";
import { Loan, LoanStatus } from "./domains/loans/types";

const book: Book = {
  isbn: "978-0-123456-78-9",
  title: "El Principito",
  author: "Antoine de Saint-Exupéry",
  genre: Genre.Fiction,
  available: true,
  copies: 3
};

const member: Member = {
  id: "M001",
  name: "Ana García",
  email: "ana@email.com",
  membershipType: MembershipType.Premium
```

Resumen

Puntos Clave

- **string, number, boolean**: Tipos primitivos básicos
- **Arrays**: Colecciones del mismo tipo (`string[]` o `Array<tipo>`)
- **Tuplas**: Arrays con tipos específicos por posición
- **Enums**: Conjuntos de valores con nombre
- **any**: Cualquier tipo (evitar si es posible)
- **void**: Para funciones sin retorno
- **null/undefined**: Valores nulos con verificación estricta
- **never**: Funciones que nunca retornan
- **Type annotations**: Anotaciones explícitas de tipo

Referencias

Recursos Adicionales

- Basic Types - TypeScript Official
- Enums - TypeScript Official
- Type Inference - TypeScript Official
- TypeScript Playground